



Gerenciador de Oficina Mecânica

Análise do Código Fonte Java, Arquitetura de Dados em Memória e Jornada Trainee

Instituição: UNEX — Centro Universitário de Excelência | Feira de Santana - BA

Curso: Bacharelado em Sistemas de Informação

Equipe de Desenvolvedores Trainee:

 Gustavo Gabriel Gomes Mauricio

 João Victor da Silva Santos Silva

 Pedro Henrique da Silva Reis Santana

 Tienrry de Jesus Carvalho

 Felipe Azevedo Brito



UNEX Feira de Santana

Excelência no Ensino Superior de Tecnologia



Mecânica Startup

Av. Visconde do Rio Branco, Baraúna



Estrutura do Projeto

CRUD Memória, TADs, Busca Linear & QA

Agenda da Apresentação

Roteiro do Tour



1. Contexto & História

A MecâniQA, o programa Trainee na UNEX e a metodologia Scrum com 5 papéis fundamentais.



2. TADs & Estruturas Base

Modelagem estilo "structs" (`Peca` e `Servico`) e arrays estáticos unidimensionais.



3. Motor CRUD & Reorganização

Inclusão no primeiro espaço vago, remoção sem buracos (shift) e controle de capacidade.



4. Busca Linear & Reuso DRY

Varredura sequencial, código `-1` de não encontrado e integração no método `Update`.



5. Tour pelo Código Java

Inspeção prática das classes `Gerenciador.java` e `Main.java` e execução das operações.



6. Gargalos & Validação QA

Limitações de memória $O(N)$, cenários de teste e as referências teóricas pesquisadas.

A Jornada Trainee MecâniQA

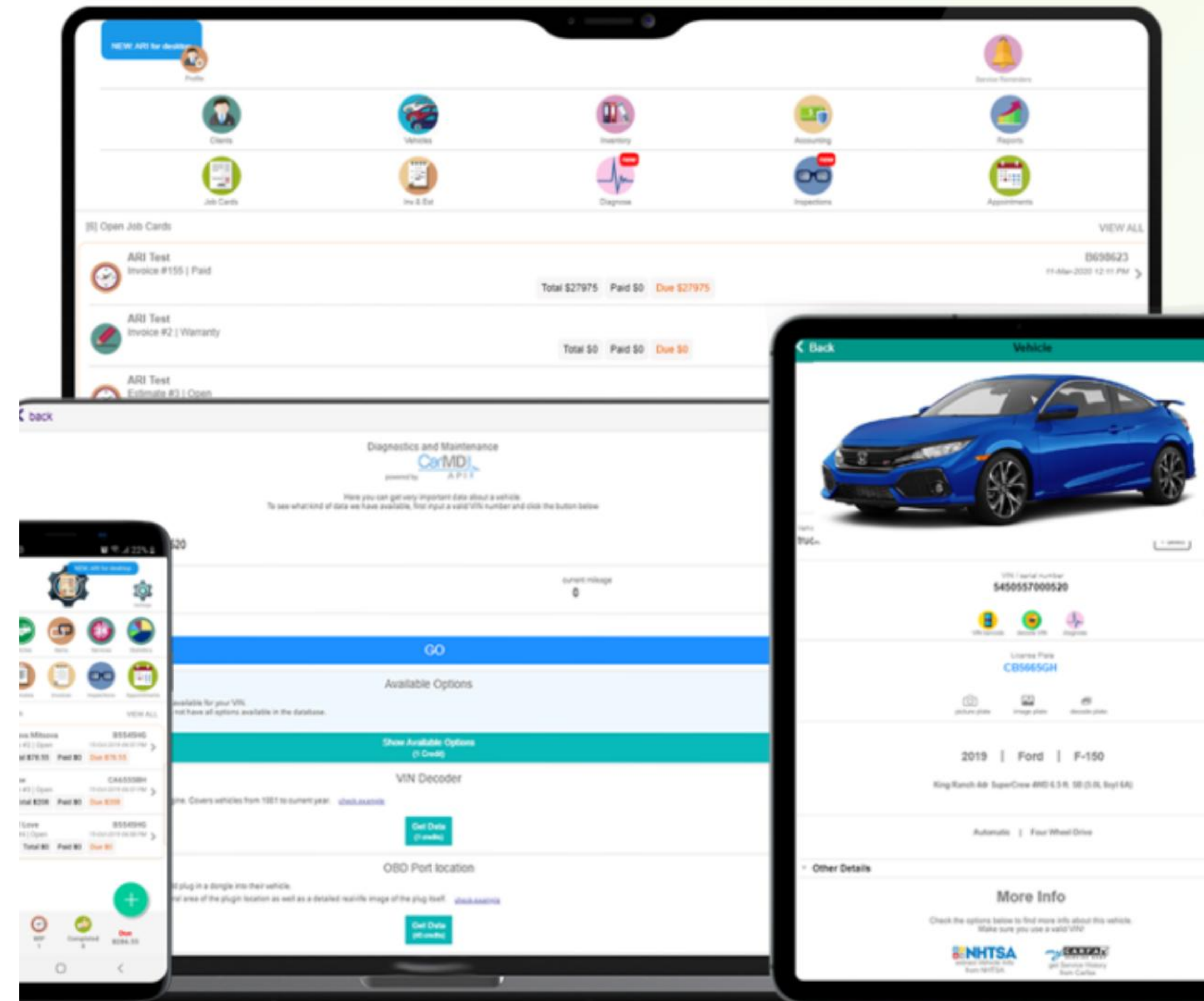
A **MecâniQA - Automotive Tech** é uma startup de Feira de Santana-BA (Baraúna) com 3 anos no mercado de gestão para auto centers e oficinas mecânicas.

No programa Trainee UNEX, assumimos o desafio de desenvolver o motor de dados do sistema central em Java sob o paradigma ágil **Scrum**.

Dinâmica de Papéis na Sprint:



- **Piloto:** Executa o código | • **Co-Piloto:** Revisor de lógica
- **QA:** Cenários de teste | • **Arquiteto:** Documentador
- **Scrum Master:** Guardião do tempo e facilitador.



Encontro 1: Definição dos TADs & Estrutura Base

Modelagem C-like

Conceito: Entidades sem Encapsulamento POO

Seguindo as diretrizes do Diretor de Engenharia, as classes foram projetadas como "structs" de C. Os atributos são diretos e públicos, sem métodos internos, getters ou setters.

- ✓ **Entidade Peça:** `codigo`, `nomePeca`, `fabricante`, `precoCusto`, `precoVenda`, `quantidadeEstoque`.
- ✓ **Entidade Serviço:** `codigo`, `nomeServico`, `descricao`, `tempoEstimadoMinutos`, `valorMaoObra`.
- ✓ **Atribuição Sequencial:** O atributo `codigo` é gerado automaticamente pelo sistema (1, 2, 3...).

```
// Peca.java – Estrutura de dados pura public class Peca { public int  
codigo; public String nomePeca; public String fabricante; public double  
precoCusto; public double precoVenda; public int quantidadeEstoque; } //  
Servico.java – Estrutura para mão de obra public class Servico { public int  
codigo; public String nomeServico; public String descricao; public int  
tempoEstimadoMinutos; public double valorMaoObra; }
```


Encontro 2: Regras de Inserção & Remoção Contígua

Estratégia Anti-Buraco

Inserção no Espaço Vago ($O(1)$)

Novos registros são inseridos imediatamente no índice apontado por `totalPecas`, dispensando varredura.

```
// Inserção contígua no final pecas[totalPecas] = novaPeca; totalPecas++;  
proximoCodigoPeca++;
```

Remoção por Reorganização ($O(N)$)

Para evitar "buracos" `null` no meio do array, os elementos à direita são deslocados uma posição para a esquerda (*Shift*).

```
// Reorganização (shift) para eliminar lacunas for (int i = indice; i <  
totalPecas - 1; i++) { pecas[i] = pecas[i + 1]; } pecas[totalPecas - 1] =  
null; totalPecas--;
```



Decisão do Time: O código identificador (`codigo`) permanece estritamente sequencial e **não é reaproveitado** após a remoção, garantindo a integridade dos registros históricos.

Encontro 3: Busca Linear & Reuso Arquitetural (DRY)

Don't Repeat Yourself

Busca Linear Sequencial

O método de busca varre o array do índice `0` até `tamanhoAtual - 1`. Como os índices válidos em Java vão de `0` a `N-1`, o retorno de erro padronizado para item não encontrado é `-1`.

```
public static int buscarPecaPorCodigo(Peca[] array, int tamanho, int cod) {  
    for (int i = 0; i < tamanho; i++) { if (array[i] != null && array[i].codigo  
    = cod) { return i; // Retorna o índice } } return -1; // Código não  
    encontrado }
```

Princípio DRY na Atualização

Em vez de reescrever um laço de repetição no método `atualizarPeca`, reutilizamos a `buscarPecaPorCodigo` para obter o índice do elemento e aplicar as alterações.

```
public static boolean atualizarPeca(int cod, String nome, ...) { int idx =  
    buscarPecaPorCodigo(pecas, totalPecas, cod); if (idx == -1) return false;  
    Peca p = pecas[idx]; p.nomePeca = nome; // Atualiza atributos return true; }
```


Tour pelo Código: Classe Gerenciador.java

Núcleo de Negócio

A classe `Gerenciador` centraliza o estado do banco em memória e os métodos estáticos para manipulação CRUD.

Atributo / Método Static	Descrição Funcional
`MAX_PECAS = 100`	Limite físico máximo do array de peças.
`MAX_SERVICOS = 50`	Limite físico máximo do array de serviços.
`totalPecas / totalServicos`	Contadores de elementos ativos armazenados.
`cadastrarPeca / cadastrarServico`	Valida capacidade, cria objeto e incrementa contador.
`listarPecas / listarServicos`	Itera do índice `0` até `total - 1` e exibe dados.
`removerPeca / removerServico`	Aplica o deslocamento à esquerda e limpa a última posição.

```
// Assinatura das estruturas estáticas public class Gerenciador {
private static final int MAX_PECAS = 100; private static final int
MAX_SERVICOS = 50; private static Peca[] pecas = new
Peca[MAX_PECAS]; private static Servico[] servicos = new
Servico[MAX_SERVICOS]; private static int totalPecas = 0; private
static int proximoCodigoPeca = 1; // CRUD Métodos Estáticos
Isolados ... }
```


Tour pelo Código: Execução em Main.java

Validação Prática

```
// Exemplo de execução das etapas em Main.java
System.out.println("##### CREATE #####"); int c1 =
Gerenciador.cadastrarPeca("Pastilha de Freio", "Bosch", 45.00, 89.90, 30);
int c2 = Gerenciador.cadastrarPeca("Filtro de Óleo", "Mann", 12.50, 29.90,
50); System.out.println("\n##### READ & BUSCA #####");
Gerenciador.listarPecas(); int idx = Gerenciador.buscarPecaPorCodigo( ... ,
c2); // Retorna índice System.out.println("\n##### DELETE & SHIFT
#####"); Gerenciador.removePeca(c2); // Remove a peça do meio
System.out.println("\n##### NOVO CADASTRO #####"); int c4 =
Gerenciador.cadastrarPeca("Vela de Ignição", "NGK", 15.00, 32.90, 100); //
c4 recebe o código 4 (ID sequencial mantido!)
```

Etapas Demonstradas no Console

- + Cadastro Inicial: Inserção de 3 Peças e 2 Serviços com geração de IDs `1`, `2` e `3`.
- 🔍 Busca e Inexistência: Busca pelo código existente e pelo código `999` (retornando `-1`).
- ✏️ Update e Delete: Alteração dos dados da peça `1` e remoção da peça `2` (do meio do array).
- 🔄 Reaproveitamento do Array: O novo item entra na posição liberta, mantendo o ID sequencial `4`.

Análise Arquitetural & Gargalos Práticos

Visão de Engenharia



101ª Peça: Estouro de Limite

O array é estático (`MAX_PECAS = 100`). A tentativa de cadastrar o 101º item é tratada com bloqueio condicional antes de disparar uma exceção de memória (`ArrayIndexOutOfBoundsException`).



Custo Computacional $O(N)$

Tanto a Busca Linear quanto o Deslocamento (Shift) na Remoção exigem tempo proporcional a N . Em grandes volumes de dados, essa arquitetura se torna ineficiente.



Evolução para Coleções Dinâmicas

Para produção, a equipe sugere evoluir de arrays fixos para `ArrayList` ou mapeamento com `HashMap`, garantindo busca em $O(1)$ sem limite predefinido.

Validação do Analista de Qualidade (QA)

Cenários de Teste

QA-01: Operação Pós-Remoção

✓ PASSOU

Cenário: Buscar, atualizar ou remover uma peça que já foi excluída.

Resultado: O sistema identificou o índice `-1` e exibiu mensagens de erro amigáveis sem travar a aplicação.

QA-02: Garantia da Sequência de IDs

✓ PASSOU

Cenário: Cadastrar nova peça após remover o código `2`.

Resultado: A nova peça recebeu o ID `4`, comprovando que o gerador sequencial não reutilizou o código deletado.

QA-03: Limite de Capacidade Máxima

✓ PASSOU

Cenário: Inserção do elemento de número `101`.

Resultado: O sistema barrou a operação e retornou `-1` com a mensagem `[ERRO] Capacidade máxima atingida`.

QA-04: Reorganização sem Buracos

✓ PASSOU

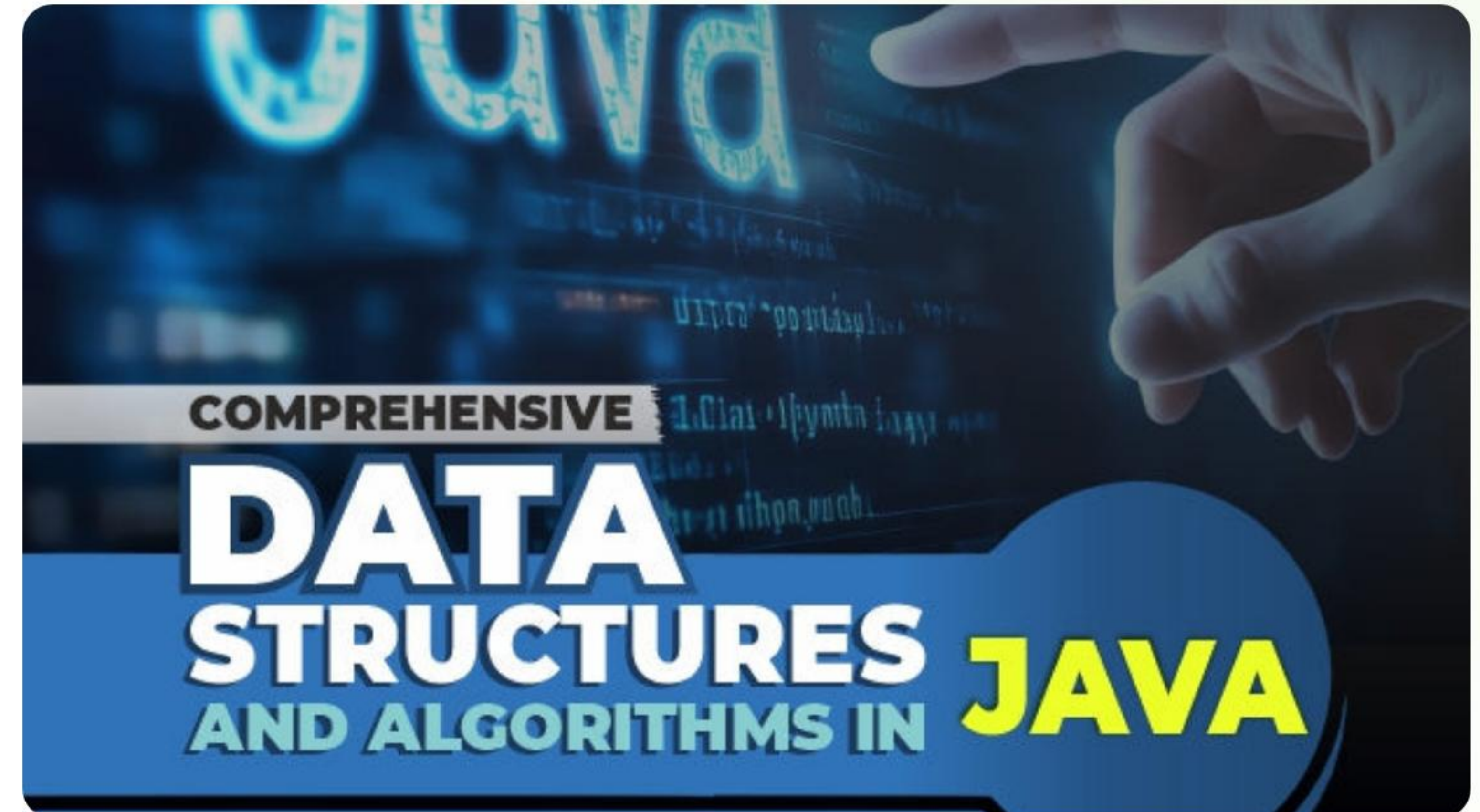
Cenário: Listar peças após a remoção do item central.

Resultado: O array manteve a contiguidade lógica sem posições intermediárias com valor `null`.

Considerações Finais & Aprendizados Trainee

Conclusões do Projeto

- 🎓 **Consolidação de Conceitos Base:** Domínio prático de Tipos Abstratos de Dados (TADs), arrays unidimensionais e operações puras em memória.
- 👥 **Colaboração Ágil Scrum:** A divisão em 5 papéis (Piloto, Co-Piloto, QA, Arquiteto e Scrum Master) garantiu eficiência e cumprimento rigoroso dos tempos de sessão.
- 🔑 **Boas Práticas de Código:** Aplicação do princípio DRY no reuso dos métodos de busca e garantia de integridade nas operações de exclusão.



Referências Bibliográficas

Links Reais PT-BR

DevMedia — Arrays em Java

Manipulação de vetores, declaração de arranjos estáticos e controle de índices em Java.
devmedia.com.br/trabalhando-com-arrays-em-java/25530

IME USP — Algoritmos de Busca Sequencial

Análise de complexidade e implementação de busca linear em estruturas contíguas na memória.
ime.usp.br/~pf/algoritmos/aulas/ordena.html

Alura — Introdução às Estruturas de Dados

Fundamentos teóricos sobre alocação em memória, vetores, listas e tempo de execução.
alura.com.br/artigos/estruturas-de-dados-introducao

UFMG DCC — Tipos Abstratos de Dados (TAD)

Conceituação de TADs, estruturas agregadas estilo 'structs' e separação de lógica e dados.
dcc.ufmg.br/~cunha/teaching/ed/tad.pdf

Guia do Scrum Oficial (Scrum Guide PT-BR)

Definição de Sprints, papéis ágeis (Scrum Master, Devs, QA) e entregas incrementais de software.
scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Portuguese-BR.pdf



Obrigado! Perguntas & Debate

Apresentação do Projeto MecâniQA – Programa Trainee UNEX
Estamos à disposição do professor e da turma para esclarecimentos técnicos.

UNEX Feira de Santana | Bacharelado em Sistemas de Informação

Image Sources



https://www.utilitymobileapps.com/wp-content/uploads/2014/11/car_repair_invoice-800x800.png

Source: www.utilitymobileapps.com



http://bpbonline.com/cdn/shop/files/2590_Epub.jpg?v=1758099935

Source: bpbonline.com